wtree — Git Worktree + Tmux Scripts

Location: `~/.dotfiles/scripts/.local/bin/`
Installed via: `stow scripts`

Overview

Four scripts that tie git worktrees to tmux sessions. Each worktree gets its own dedicated tmux session, pre-split into a terminal pane (left) and a Claude Code pane (right), so you can jump between parallel branches without losing context.

Script	Purpose
`wtree`	Create a new branch, worktree, and tmux session
`wtree-attach`	Attach to an existing worktree session (recreates if needed)
`wtree-ls`	List all worktrees and whether their tmux sessions are active
`wtree-rm`	Remove a worktree, delete its branch, and kill its tmux session

Tmux sessions are named `<repo-name>-<branch-name>`. Worktrees are created as siblings to the repo root at `../worktrees/<branch>/`.

Quick Start

```
# From inside any git repo:
wtree feature-auth          # Create worktree + session from HEAD
wtree fix-login main        # Create worktree + session based on main

wtree-ls                    # See all worktrees and session status
wtree-attach feature-auth   # Reattach to an existing session
wtree-rm feature-auth       # Clean up when done
```

Script Details

`wtree <branch-name> [base-branch]`

Creates a new git branch, checks it out as a worktree at `../worktrees/<branch>/`, then opens a tmux session with two panes:

- **Left pane** — shell, starting in the worktree directory
- **Right pane** — Claude Code (`claude`), auto-started

```
wtree feature-auth          # base defaults to HEAD
wtree fix-bug main          # base on the main branch
```

> **Note:** Errors if the branch or worktree directory already exists. Use `wtree-attach` in that case.

`wtree-attach <branch-name>`

Attaches to the tmux session for an existing worktree. If the session has been killed (e.g. after a reboot) but the worktree still exists, it recreates the split-pane session and restarts Claude Code.

```
wtree-attach feature-auth
```

`wtree-ls`

Lists every git worktree for the current repo, showing the filesystem path and whether its tmux session is currently active. Also shows all live tmux sessions matching the `<repo-name>-*` naming pattern.

```
wtree-ls
```

`wtree-rm <branch-name>`

Removes a worktree completely. Prompts for confirmation, then:

1. Kills the tmux session (if active)
2. Runs `git worktree remove --force`
3. Removes the empty `../worktrees/` directory if no worktrees remain

```
wtree-rm feature-auth
```

> **Note:** This does **not** delete the git branch itself — only the worktree checkout. Delete the branch separately with `git branch -d feature-auth` if desired.

Typical Workflow

```
# Start a new feature
wtree feature-payments

# Left pane: write code, run tests
# Right pane: Claude Code is already running

# Switch to a different task (detach with Ctrl+Space d)
wtree another-task

# Come back later
```

```
wtree-attach feature-payments

# See everything at a glance
wtree-ls

# Clean up after merging
wtree-rm feature-payments
git branch -d feature-payments   # optional: delete the branch too
```

Installation

Scripts are managed via GNU Stow from the dotfiles repo:

```
cd ~/.dotfiles
stow scripts            # creates ~/.local/bin/wtree* symlinks
```

Ensure `~/.local/bin` is in your `$PATH` (it is by default in the dotfiles `.zshrc`).

Naming Conventions

Thing	Pattern	Example
Worktree path	`../worktrees/<branch>/`	`../worktrees/feature-auth/`
Tmux session	`<repo>-<branch>`	`myapp-feature-auth`
Special chars in branch name	Replaced with `_`	`feat/auth` → `feat_auth`